

[C.T.]

Linked List

Name: Sunendu Bena.

Roll NO: UG1/04/DT CSE/2025/024

1. Data and a pointer to the next node. (b)

2. NULL (c)

3. self-referential structure. (b)

4. head == NULL (b)

5. newnode $\rightarrow$ next = head; head = newnode; (b)

6. temp = head;

head = head $\rightarrow$ next;

free(temp);

Ans) - (b)

Sunendu Bena. Roll - 024

7. Deleting the head node. (d)

8. n (b)

9. Copying the data of p $\rightarrow$ next into P, then deleting

p $\rightarrow$ next. (c)

Sunendu Bena. Roll - 24

11. while (p != NULL) (a)

12. p → next = NULL (b)

13. Two pointers advancing at different rates. (c)

14. Three (c)

15. Exactly one node (b)

16. p = (struct node*) malloc (sizeof (struct node)); (b)

Sunendy Bena. Roll-024

17. Avoid special case handling for insertion and deletion of the head (b)

18. The first node. (c)

19. It cannot store duplicate values. (a)

20. Loss of access to all nodes, causing a memory leak. (b)

21. The last node (c)

22. Nodes may be scattered anywhere in memory. (a)

```
struct Node {
```

```
    int data;
```

```
    struct Node *next;
```

```
};
```

```
26. struct Node* reverseList(struct Node* head) {
```

```
    struct Node* prev = NULL;
```

```
    struct Node* curr = head;
```

```
    struct Node* next = NULL;
```

```
    while (curr != NULL) {
```

```
        next = curr->next;
```

```
        curr->next = prev;
```

```
        prev = curr;
```

```
        curr = next;
```

```
    }
```

```
    return prev;
```

24. 4

```
void insert_end(struct node **head, int x)
```

```
{  
    struct node *temp = (*head);  
    if (temp == NULL)
```

```
    {  
        (*head) = malloc(sizeof(struct node));  
        (*head) = temp;
```

```
        temp->next = NULL;  
        temp->data = x;
```

```
    }  
    else
```

```
    {  
        struct node *temp = (*head);  
        while (temp->next != NULL)
```

```
        {  
            temp = temp->next;
```

```
        }  
        temp->next = malloc(sizeof(struct node));  
        temp->next->data = x;  
        temp->next->next = NULL;
```

```
    }  
}
```

```
int main()  
{  
    struct node *head = NULL;  
    insert_end(&head, 10);  
    insert_end(&head, 20);  
    insert_end(&head, 30);  
    insert_end(&head, 40);  
    insert_end(&head, 50);  
    display(head);  
    return 0;  
}
```